



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

6. Ausnahmebehandlung (Exceptions)



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.1 Einleitung & Fehlercodes



Einleitung

- Arten von Fehlern bei der Programmierung
 - **Syntax-Fehler**
 - Werden vom Compiler erkannt; erst nach deren Behebung kann das Programm übersetzt und gestartet werden
 - **Logische / semantische Fehler**
 - Werden z.B. durch fehlgeschlagene Testcases erkannt; können i.a. mit Hilfe eines Debuggers behoben werden
 - **Laufzeitfehler / Exceptions**
 - Oft liegt die Fehlerquelle außerhalb des Programms, z.B. falsche Benutzereingaben, fehlende Dateien usw.; diese „Ausnahmen“ müssen im Programm behandelt und abgefangen werden
- Im folgenden werden die Fehler / Ausnahmen betrachtet, die zur Laufzeit auftreten.
- Java bietet hierfür das Konzept der **Exceptions** (Ausnahmen) an.

Fehler vs. Ausnahmen

- Fehler und Ausnahmen unterbrechen die normale Programmausführung und stellen ein **nicht geplantes Ereignis** dar
 - Java ermöglicht es, solche nicht geplanten Ereignisse abzufangen und zu behandeln (als Alternative zum Programmabsturz).
- **Fehler** treten in der virtuellen Maschine von Java auf und sind **nicht reparierbar**.
 - Beispiele: Kein Speicher mehr verfügbar, zu hoher Aufrufstapel, fehlende Programmbibliotheken ...
- **Ausnahmen** werden von der virtuellen Maschine oder vom Programm selbst ausgelöst und können meist behandelt werden, um die **Normalsituation** wiederherzustellen.
 - Beispiele: Dereferenzierung von null, Division durch 0, Schreib- / Lesefehler (Dateien)



Hier ist nichts mehr zu machen.



Aufwischen und neu einschenken.



Fehlercodes - Traditionelle Fehlerbehandlung

- Bei der Implementierung eines Stack durch ein Array können zwei Fehler auftreten:
 1. Einfügen in einen vollen Stack
 2. Lesen aus einem leeren Stack
- Beim Lesen aus dem Stack kann der Fehler über einen speziellen Rückgabewert (Fehlercode) der Methode gemeldet.
$$q.pop() = -1$$
- Problem: -1 gehört zum normalen Wertebereich der Stack-Elemente
- Fachwort dafür: **In-Band Signaling**



Fehlercodes - Traditionelle Fehlerbehandlung

- Beim Einfügen in eine vollen Stack wird einfach „nichts“ getan.
`push(x)`
- Der Fehler wurde also nicht gemeldet
- Java-Syntax: Methoden müssen immer oder nie einen Rückgabewert haben.
- Lösungsmöglichkeit: Fallunterscheidungen zum Abfragen von Fehlercodes durchführen und Code zum Behandeln der Fehler enthalten.
- Ggf. müsste der Fehler an eine aufrufende Methode weitergegeben werden, und dort verarbeitet werden. . .
- **Fazit:** Fehler sollten immer gemeldet werden! Fehlerbehandlung sollte von Programmlogik getrennt sein.



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.2 Konzept Java Exceptions



Motivation

- Die Behandlung von Ausnahmen mittels Exceptions soll die genannte Vorgehensweise nicht in jedem Fall ersetzen.
- Exceptions erlauben jedoch,
 1. das Auftreten der Ausnahme und deren Behandlung **räumlich im Code zu trennen**,
 2. die Behandlung der Ausnahme **dort vorzunehmen, wo sie sinnvoll** ist, d.h. ggf. auch „höher“ in der Hierarchie der aufrufenden Funktionen, ohne dafür kaskadierende und geschachtelte `if`-Abfragen implementieren zu müssen,
 3. den Code zugunsten einer **besseren Übersicht** nicht durch eine aufwändige Ausnahmebehandlung unterbrechen zu müssen,
 4. verschiedene Ausnahmetypen (über Objekte in einer geeigneten Klassenhierarchie) voneinander zu **unterscheiden**.



Konzepte des Exception Handling in Java

- Das Exception Handling (Mechanismus zur Ausnahmebehandlung) beruht auf 3 Konzepten:
 1. Exception `throw`
 2. Geschützter Block `try`
 3. Exception Handler `catch`



1. Exception

- Eine Ausnahme wird signalisiert durch das Auslösen einer Exception mittels `throw`
- Das Werfen einer Ausnahme kann selbst implementiert sein; Ausnahmen können aber auch von verwendeten Methoden anderer Klassen ausgelöst werden
- Wenn eine Ausnahme ausgelöst wird, wird der Programmablauf unterbrochen und an der Stelle fortgesetzt, an der ein passender Ausnahmebehandler vorgefunden wird
- Eine Exception ist ein Objekt der Klasse `Exception` oder einer abgeleiteten Klasse von `Exception`

```
double zaehler, nenner;  
// ... z.B. zaehler, nenner einlesen  
if (nenner == 0.0) {  
    throw new Exception(); // Ausnahme wird geworfen  
} else {  
    double ergebnis = zaehler / nenner;  
}
```





2. Geschützter Block

- Programmcode, in dem eine Ausnahme ausgelöst werden kann, wird in einen geschützten Block gekapselt, der mit dem Schlüsselwort `try` markiert wird.
- Der so geschützte Block überwacht mögliche Ausnahmen und ruft ggf. speziellen Programmcode zur Ausnahmebehandlung auf

```
try {  
    // geschützter Block; Programmcode, der eine Ausnahme werfen kann  
}
```





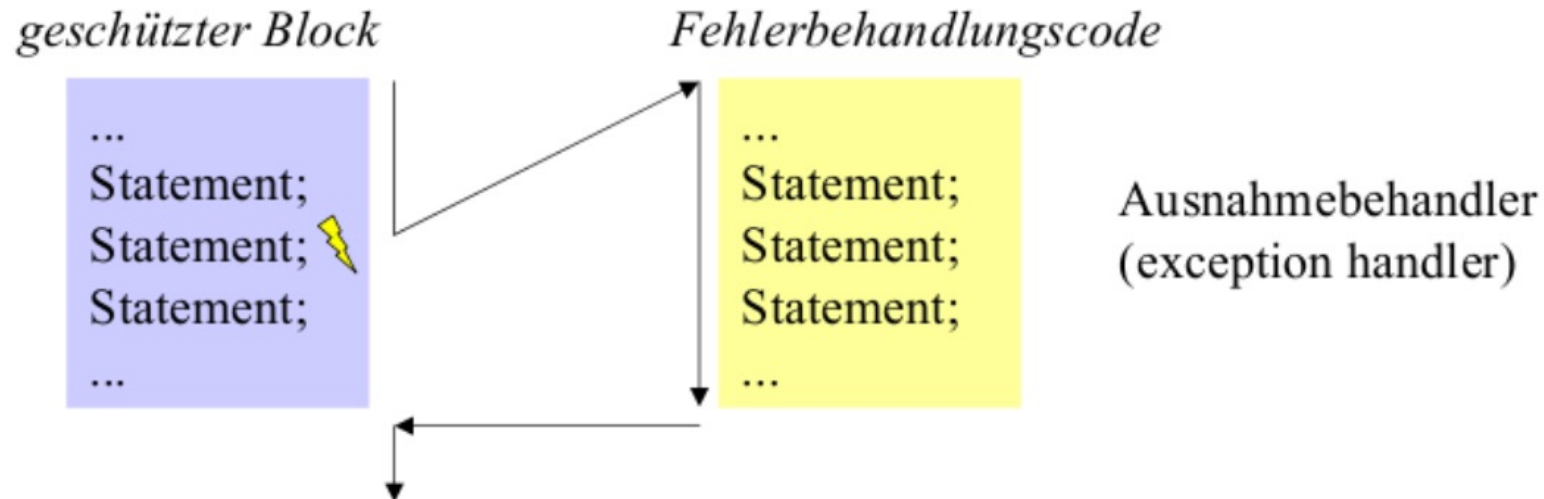
3. Exception Handler

- Ein geschützter Block hat einen oder mehrere Exception Handler.
- Das Schlüsselwort für einen Exception Handler ist `catch`.
- Ein Exception Handler ist eine Anweisungsfolge, in der auf eine Ausnahme reagiert wird.
- Jeder Exception Handler fängt eine geworfene, zu ihm passende Ausnahme ab und behandelt sie.

```
try {  
    // geschützter Block; Programmcode, der eine Ausnahme werfen kann  
}  
catch (Exception e) {  
    // Exception Handler; Programmcode zum Behandeln der Ausnahme  
}  
// Es geht ganz normal weiter, denn die Ausnahme wurde behandelt
```



Überblick: Exception Konzept



```
try {  
    // geschützter Block; Programmcode, der eine Ausnahme werfen kann  
}  
catch (Exception e) {  
    // Exception Handler; Programmcode zum Behandeln der Ausnahme  
}  
// Es geht ganz normal weiter, denn die Ausnahme wurde behandelt
```





Programmieren

6. Ausnahmebehandlung (Exceptions)

6.3 Arten von Ausnahmen in Java



Arten von Ausnahmen

- **Geprüfte Ausnahmen**

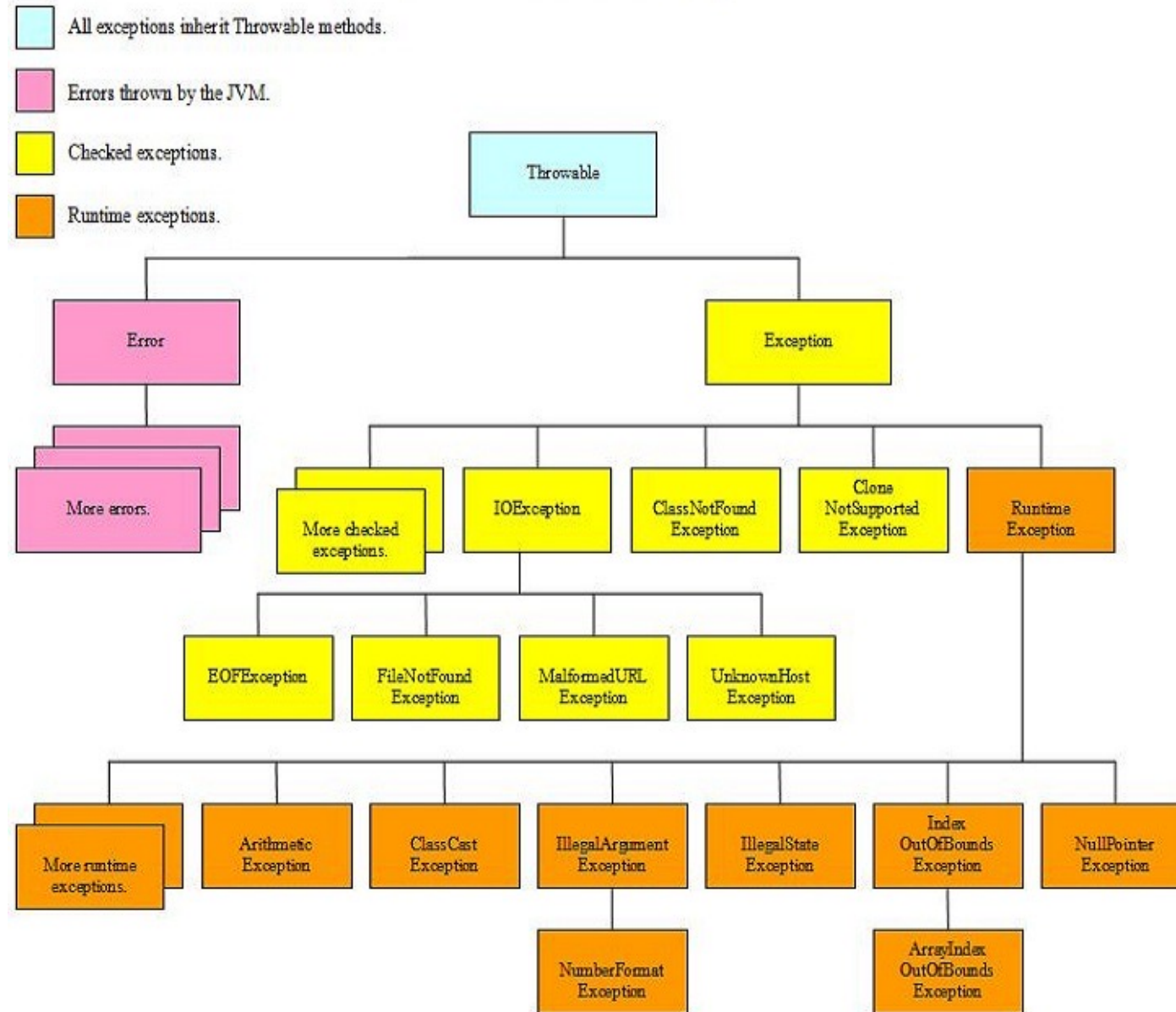
- auch kontrollierte Ausnahmen oder Benutzerausnahmen; Englisch: **checked exceptions**
- Müssen behandelt oder aber weitergeleitet werden (der Compiler prüft, ob eine entsprechende Behandlung vorhanden ist)
- Ursache: Unwahrscheinliches, aber prinzipiell mögliches Ereignis; Beispiel: eine zu öffnende Datei existiert nicht oder ist leer

- **Ungeprüfte Ausnahmen**

- auch nicht kontrollierte Ausnahmen oder Laufzeitausnahmen genannt; Englisch: **unchecked exceptions**, runtime exceptions
- Können während der normalen Operation (Laufzeit) der JVM überall im Code auftreten
- Können behandelt werden, müssen aber nicht (dann: Programmende); der Compiler beachtet diese Ausnahmen nicht (daher ungeprüft)
- Ursache: i.a. Programmierfehler / Schwachstellen im Code (z.B. Division durch Null, Zugriff auf nicht existierende Feldelemente, ...)



Exception Hierarchy





Geprüfte Ausnahmen

- **Geprüfte Ausnahmen** sind Instanzen (einer Unterklasse) von `Exception` (aber nicht von `RuntimeException`) und werden durch eine `throw`-Anweisung ausgelöst.
- Die Java-Bibliothek enthält vordeklarierte Ausnahmen für häufige Ausnahmearten (z.B. `java.io.IOException`).
- Man kann in Programmen eigene Ausnahmen definieren, auslösen und behandeln.
- Alle geprüften Ausnahmen müssen in einem Exception Handler abgefangen werden, sonst lässt sich das Programm nicht übersetzen (Compilerprüfung).



Ungeprüfte Ausnahmen

- **Ungeprüfte Ausnahmen** sind Instanzen (einer Unterklasse) von `RuntimeException` und werden durch eine `throw`-Anweisung oder auch automatisch ausgelöst, wenn ein Programm eine illegale Instruktion ausführt
 - `ArithmeticException`: ausgelöst bei Division durch 0.
 - `NullPointerException`: ausgelöst, wenn über eine Referenzvariable auf ein Objekt zugegriffen werden soll, die Referenz aber `null` ist.
 - `ArrayIndexOutOfBoundsException`: ausgelöst, wenn bei einem Zugriff auf ein Arrayelement der Index außerhalb des Arrays liegt.
- Laufzeitausnahmen müssen nicht unbedingt in einem Exception Handler behandelt werden. In diesem Fall bricht das Programm ab mit einer Fehlermeldung. Mit Exception Handler läuft das Programm nach Abarbeiten des Ausnahmebehandlungscode weiter.



Klasse Exception

- Ausnahmen sind in Java Objekte
- Sie bieten Methoden an, um auf Informationen zu der aufgetretenen Ausnahme zuzugreifen.
- Die Klasse `java.lang.Exception` besitzt diverse Unterklassen:
 - `RuntimeException` (mit Unterklassen `ArithmeticException`, `NullPointerException`, etc.)
 - `IOException`, `InterruptedException` etc. als vordeklarierte Ausnahmen
- Selbstdefinierte Ausnahmen (checked oder unchecked) müssen als Unterklassen von `Exception` realisiert werden.
- Falls sie als Unterklasse von `RuntimeException` realisiert werden, handelt es sich um eine unchecked exception



Schnittstelle der Basisklasse Exception

- Die Klasse `Exception` bietet die folgende Schnittstelle, die auch von einer selbst definierten Ausnahme geerbt wird (hier Auszüge)
 - `Exception()` Konstruktor ohne Parameter
 - `Exception(String message)` Konstruktor mit einem Informations-String als Parameter
 - `String toString()` kurze Beschreibung des Exception-Objekts
 - `void printStackTrace()` gibt Kette der Methodenaufrufe aus, die zu der Exception geführt haben.
 - ...



Eigene Ausnahmen

- Wenn wir eine Ausnahme behandeln wollen, für den es noch keine vordefinierte Exception-Klasse gibt, müssen wir eine eigene Exception implementieren.
- Dazu gehören die folgenden Elemente:
 - Ausnahme: Klasse, die die spezielle Ausnahme darstellt.
 - Geschützter Block: Kapselung aller Stellen, die die Ausnahme auslösen können. Dabei wird die Ausnahme entweder direkt im try-Block oder von einer im try-Block aufgerufenen Methode ausgelöst.
 - Exception Handler: Block, der die Ausnahme behandelt.



Beispiel Stack

- Wir definieren eine Klasse `OverflowException`.
- Deren Objekte werden verwendet, um einen Stapelüberlauf anzuzeigen.
- Das Element, das sich nicht ablegen lässt, wird der Exception übergeben

```
public class OverflowException extends Exception {  
    private int overflowElement;  
  
    public OverflowException(int x) { this.overflowElement = x; }  
  
    public int getOverflowElement( ) { return overflowElement; }  
}
```



```
public class UnderflowException extends Exception { ...
```





Beispiel Stack



```
public class ArrayStack implements Stack {  
  
    private int[] elements = null; // Elemente  
    private int num = 0; // aktuelle Anzahl  
  
    public ArrayStack(int size) {  
        elements = new int[size]; // Stack mit definierter Kapazität  
    }  
  
    public void push(int x) throws OverflowException {  
        if (num == elements.length) {  
            throw new OverflowException(x);  
        }  
        elements[num++] = x;  
    }  
  
    public int pop() throws UnderflowException {  
        if (num == 0) { // Stack ist leer  
            throw new UnderflowException();  
        }  
        int o = elements[--num];  
        return o;  
    }  
}
```




Beispiel Stack



```
public static void main(String[] args) {  
    Stack stack = new ArrayStack(1);  
    try {  
        stack.push(2); // fügt 2 dem Stapel hinzu  
        stack.push(3); // löst OverflowException aus  
        // ... wird nicht mehr ausgeführt  
    } catch (OverflowException e) {  
        System.out.println( "Stack voll: Das Element " + e.getOverflowElement() +  
                             " konnte nicht auf den Stack gelegt werden.");  
    }  
    try {  
        stack.pop(); // entnimmt 2 dem Stapel  
        stack.pop(); // löst UnderflowException aus  
    } catch (UnderflowException e) {  
        System.out.println("Stack leer: Kann kein Element entnehmen.");  
    }  
}
```



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.4 Ausnahmebehandler



Ausnahmebehandler

- Ein Ausnahmebehandler / Exception Handler ist eine Anweisungsfolge zur Ausnahmebehandlung.
 - ähnelt einer Methode und hat einen Parameter
 - Parameter ist Objekt der aufgetretenen Ausnahme.
 - Es darf mehrere Exception Handler nach einem try-Block geben (mehrere catch-Blöcke hintereinander).



Ausnahmebehandler

- Bei mehreren `catch`-Blöcken
 - Die Wahl des Ausnahmebehandlers erfolgt anhand des Parametertyps der `Exception` (des `Exception`-Objekts).
 - Die `catch`-Klauseln werden der Reihe nach geprüft.
 - Die erste passende `catch`-Klausel wird ausgewählt.
 - Spezifische `catch`-Klauseln müssen vor allgemeineren stehen (der Compiler prüft auf korrekte Reihenfolge von der speziellen zur allgemeineren `catch`-Klausel).
- Nach den eigenen `catch`-Klauseln wird nach passenden `catch`-Klauseln des aufrufenden `try`-Blocks geschaut.
- Programm setzt Ausführung nach dem `try`-Block fort.



Ausnahmebehandler

- Fragen:
 - Was passiert beim Auslösen einer Exception?
 - Was wäre bei einer anderen Reihenfolge der catch-Klauseln?

```
try {  
    stack.push(1);  
    stack.pop();  
} catch (OverflowException e) {  
    System.out.println(e);  
} catch (UnderflowException e) {  
    System.out.println(e);  
} catch (NullPointerException e) {  
    System.out.println(e);  
} catch (Exception e) {  
    System.out.println(e);  
}
```





Programmieren

6. Ausnahmebehandlung (Exceptions)

6.5 Auslösen einer Ausnahme



Auslösen einer Ausnahme

- Eine geprüfte Ausnahme wird durch eine throw-Anweisung ausgelöst.
- Das Objekt der Ausnahmeklasse wird dann einem passenden Exception Handler „zugeworfen“

```
public void push(int x) throws OverflowException {  
    if (num == elements.length) {  
        throw new OverflowException(x);  
    }  
    elements[num++] = x;  
}  
  
public int pop() throws UnderflowException {  
    if (num == 0) { // Stack ist leer  
        throw new UnderflowException();  
    }  
    int o = elements[--num];  
    return o;  
}
```





Wirkung einer throw-Anweisung

- Was bewirkt die `throw`-Anweisung?
 - In allen dynamisch umgebenden `try`-Blöcken des gerade laufenden `try`-Blocks wird der Reihe nach ein passender Exception Handler gesucht.
 - Zuerst im `try`-Block der zuletzt aufgerufenen Methoden, dann die aufrufende usw.
 - In den `catch`-Blöcken von oben nach unten
 - Die laufende Anweisungsfolge wird unterbrochen.
 - Der ausgewählte Exception Handler wird gestartet und erhält das Exception-Objekt als Parameter.
 - Nach Abarbeiten der `catch`-Klausel wird die Ausführung des Programms hinter der `try`-Anweisung fortgeführt, zu der die `catch`-Klausel gehört.



Beispiel

- Was passiert beim Auslösen von Exceptions E1 bzw. E2 in Methode m3?

```
void m1() {  
    try {  
        // ...  
        m2();  
        // ...  
    } catch (E1 e) {  
        // ...  
    } catch (E2 e) {  
        // ...  
    }  
    // ...  
}
```

```
void m2() throws E1 {  
    // ...  
    m3();  
    // ...  
}
```

```
void m3() throws E1 {  
    try {  
        if (b) {  
            // ...  
            throw new E1();  
            // ...  
        } else {  
            // ...  
            throw new E2();  
            // ...  
        }  
    } catch (E2 e) { /*...*/ }  
}
```



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.6 Die finally-Klausel



Finally

- Nach einer Exception wird die Ausführung nach dem `try`-Block fortgesetzt, zu dem die ausgewählte `catch`-Klausel gehört.
- Was passiert mit Code-Teilen, die in jedem Fall ausgeführt werden sollten?
- Beispiel: Geöffnete Dateien sollten geschlossen werden.



Finally

- Anweisungen der `finally`-Klausel werden immer am Ende eines `try`-Blocks ausgeführt, egal, ob eine Ausnahme auftrat oder nicht.
- Wird nach einem `try`-Block keine passende `catch`-Klausel gefunden, wird eine vorhandene `finally`-Klausel ausgeführt, bevor die Suche nach einer `catch`-Klausel im nächstäußeren `try`-Block fortgesetzt wird.



Beispiel

- Was passiert beim Auslösen von Exceptions E1 bzw. E2 in Methode m3?

```
void m1() {  
    try {  
        // ...  
        m2();  
        // ...  
    } catch (E1 e) {  
        // ...  
    } catch (E2 e) {  
        // ...  
    } finally {  
        // ... A  
    }  
    // ...  
}
```

```
void m2() throws E1 {  
    // ...  
    m3();  
    // ...  
}
```

```
void m3() throws E1 {  
    try {  
        if (b) {  
            // ...  
            throw new E1();  
            // ...  
        } else {  
            // ...  
            throw new E2();  
            // ...  
        }  
    } catch (E2 e) { /* .. */  
    } finally { /* .. B */ }  
}
```



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.7 Spezifikation von Ausnahmen im Methodenkopf



Ausnahme an die aufrufende Methode weiterleiten

- Beispiel: Ausnahme beim Öffnen einer Datei
 - Wenn die Datei nicht existiert, ist das eine Ausnahme
 - Behandlung meist nur in der aufrufenden Methode adäquat möglich
 - `openFile` enthält also keinen `try`-Block, sondern überlässt das Exception Handling der aufrufenden Methode

```
public File openFile(String name) {  
    Path p = Paths.get(name);  
    if (p.toFile().exists()) {  
        return p.toFile();  
    } else {  
        throw new FileNotFoundException();  
    }  
}
```





Exception Handling in aufrufender Methode

- Woher weiß die aufrufende Methode, dass `openFile` eine Exception auslösen kann?
 - `openFile` könnte eine Methode aus einer Bibliothek sein, d.h. wir haben keinen Zugriff auf den Code.
- Lösung
 - Man spezifiziert im Kopf jeder Methode alle Ausnahmen, die von der Methode ausgelöst werden können und nicht abgefangen werden.
 - Das ist für alle Ausnahmen erforderlich, die vom Compiler geprüft werden → checked exceptions (Benutzerausnahmen)
 - Unchecked (runtime-)exceptions sind ungeprüft, d.h. sie dürfen ohne Ankündigung geworfen werden.



Ausnahme an die aufrufende Methode weiterleiten

- Es können unterschiedliche Ausnahmen in `openFile` ausgelöst werden
- Die `throws`-Klausel im Methodenkopf gehört wie die Parameter der Methode zur Schnittstelle der Methode und erscheint in jeder Dokumentation.
- Damit weiß die aufrufende Methode, welche Exceptions bei einem Aufruf auftreten können.

```
public File openFile(String name)
    throws FileNotFoundException,
        AccessDeniedException {
    Path p = Paths.get(name);
    if (p.toFile().exists()) {
        return p.toFile();
    } else {
        throw new FileNotFoundException();
    }
}
```



Ausnahme an die aufrufende Methode weiterleiten

- Die Ausnahmen werden in der aufrufenden Methode behandelt

```
void m() {  
    try {  
        File file = openFile("hello.txt");  
    } catch (AccessDeniedException e) {  
        e.printStackTrace();  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```





Ausnahme an die aufrufende Methode weiterleiten

- Auch aufrufende Methoden können Exceptions weiterleiten

```
void m2() throws FileNotFoundException, AccessDeniedException {  
    File file = openFile("hello.txt");  
    // ...  
}
```



- Wenn Exceptions auch in der aufrufenden Methode weitergeleitet werden sollen, muss diese ebenfalls eine throws-Klausel enthalten.
- Unchecked exceptions, z.B. eine `NullPointerException`, muss man nicht im Methodenkopf spezifizieren.
- Werden unchecked exceptions in keiner Methode des Aufrufstacks behandelt, wird im Falle des Auftretens einer Ausnahme eine Standardfehlermeldung ausgegeben und das Programm abgebrochen.



Beispiel

- Ergänzung von throws

```
void m1() {  
    try {  
        // ...  
        m2();  
        // ...  
    } catch (E1 e) {  
        // ...  
    } catch (E2 e) {  
        // ...  
    } finally {  
        // ... A  
    }  
    // ...  
}
```

```
void m2() throws E1 {  
    // ...  
    m3();  
    // ...  
}
```

```
void m3() throws E1 {  
    try {  
        if (b) {  
            // ...  
            throw new E1();  
            // ...  
        } else {  
            // ...  
            throw new E2();  
            // ...  
        }  
    } catch (E2 e) { /*...*/  
    } finally { /* .. B */ }  
}
```



Beispiel Stack

- Die Methoden `push` und `pop` der Klasse `Stack` lösen ggf. eine Ausnahme aus und leiten sie mit einer `throws`-Klausel weiter, anstatt sie direkt zu behandeln

```
public void push(int x) throws OverflowException {   
    if (num == elements.length) {  
        throw new OverflowException(x);  
    }  
    elements[num++] = x;  
}  
  
public int pop() throws UnderflowException {  
    if (num == 0) { // Stack ist leer  
        throw new UnderflowException();  
    }  
    int o = elements[--num];  
    return o;  
}
```



Programmieren

6. Ausnahmebehandlung (Exceptions)

6.8 Exceptions seit Java 7



Schreibarbeit sparen mit „multi-catch“

- In Java-Programmen müssen oft viele Standard-Exceptions behandelt werden
- Wenn mehrere Exception-Typen gemeinsam behandelt werden können, kann ein **multi-catch** verwendet werden

```
void m() {  
    try {  
        File file = openFile("hello.txt");  
    } catch (AccessDeniedException e) {  
        e.printStackTrace();  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```



```
void m() {  
    try {  
        File file = openFile("hello.txt");  
    } catch (AccessDeniedException |  
             FileNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```





Automatisches Schließen von Ressourcen

- Interface `AutoCloseable`: Man kann alternativ zu try-catch ein **try-with-resources** verwenden.
- Das Laufzeitsystem sorgt dann dafür, dass automatisch alle im Header des `try` geöffneten Ressourcen mit der in `AutoCloseable` definierten `close()`-Methode geschlossen werden, bevor eine `finally`- oder `catch`-Klausel ausgeführt wird.

```
// ...
try (FileReader file = new FileReader("file.txt");
    BufferedReader br = new BufferedReader(file)) {
    // ...
} catch (IOException e) {
    // ...
}
// ...
```





Exceptions - Best Practices

- Exceptions sollen nur dann geworfen werden, wenn die Methode die Situation nicht selbst behandeln kann, keinesfalls als eine Art von Rückgabe.
- Der Name einer Exception sollte sich sinnvoll an der Methode orientieren, die sie wirft, bzw. die Art der Ausnahme sinnvoll charakterisieren.
- Checked exceptions beziehen sich u.a. auf den Fall, dass eine Methode nicht leisten kann, was der Name verspricht, z.B. `FileNotFoundException`.
- Eine Checked exception ist nur sinnvoll, wenn der ursprüngliche Aufrufer mit dem Ergebnis, d.h. der teilweisen Ausführung der Methode sowie des `finally`-Statements und der Behandlung der Exception, sinnvoll weiterarbeiten kann. Ansonsten verwenden Sie unchecked exceptions.
- “If a client can reasonably be expected to recover from an exception, make it a checked exception. If a client cannot do anything to recover from the exception, make it an unchecked exception.”
(<https://docs.oracle.com/javase/tutorial/essential/exceptions/runtime.html>)



Checked Exceptions und Vererbung

- Beim Überschreiben einer Methode in einer abgeleiteten Klasse darf die Signatur gegenüber der überschriebenen Methode nur eingeschränkt, jedoch nicht erweitert werden
- Exception wie ein Rückgabetyp (Kovarianz)

```
public class A {  
    public void tuEtwas() throws IOException { /* ... */ }  
}
```



```
public class B extends A {    // Compile-Fehler, hinzufügen nicht erlaubt  
    public void tuEtwas() throws IOException, SQLException { /* ... */ }  
}
```





Checked Exceptions und Vererbung

```
public class A {  
    public void tuEtwas() throws IOException {/* ... */}  
}
```



```
public class B extends A {    // Compile-Fehler, hinzufügen nicht erlaubt  
    public void tuEtwas() throws IOException, SQLException {/* ... */}  
}
```



```
public class B extends A {    // Ok  
    public void tuEtwas() {/* ... */}  
}
```



```
public class B extends A {    // Ok, weil Unterklasse von IOException  
    public void tuEtwas() throws FileNotFoundException {/* ... */}  
}
```





Exceptions - Übung

- Entwickeln Sie eine Klasse Kontakt mit den Attributen `vorname`, `nachname` und `telnr`.
- Die Methode `setNachname` soll eine Exception `InvalidNameException` werfen, wenn der übergebene String einen numerischen Wert darstellt.
- Die Methode `setVorname` soll eine Exception `InvalidNameException` werfen, wenn der übergebene String nicht nur Buchstaben enthält.
- Überlegen Sie sich auch eine Prüfung für Telefonnummer.